

ASSIGNMENT 7.3

ShaikSabina

2303A51303

Batch – 05

TASK – 01 :

Prompt : Identify and fix the syntax error in a Python function add(a, b) where the function definition is incorrect. Correct the code and briefly explain what the error was and how it was fixed with example usage.

Code :

```
assignment7.3.py > ...
1  #Identify and fix the syntax error in a Python function add(a, b) where the function definition is incorrect.
2  #Correct the code and briefly explain what the error was and how it was fixed with example usage.
3  # Original code with syntax error
4  '''def add(a, b)
5      return a + b'''
6  # Corrected code
7  def add(a, b):
8      return a + b
9  # Explanation:
10 # The error in the original code was a missing colon (:) at the end of the function definition line.
11 # In Python, function definitions must end with a colon to indicate the start of the function body.
12 # By adding the colon, the syntax error is resolved, and the function can be executed correctly.
13 # Example usage
14 result = add(3, 5)
15 print(result) # Output: 8
```

Output :

```
PS C:\Users\devi\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/devi/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
File "c:\Users\devi\OneDrive\Desktop\AI Assisted Coding\assignment7.3.py", line 4
def add(a, b)
      ^
SyntaxError: expected ':'
PS C:\Users\devi\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/devi/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
8
PS C:\Users\devi\OneDrive\Desktop\AI Assisted Coding> |
```

Explanation :

The original function had a syntax error because the colon (:) was missing at the end of the function definition. In Python, the colon is required to start the function body. Adding the colon fixes the error, allowing the function to run correctly. The example shows the function successfully adding two numbers and printing the result.

TASK – 02:

Prompt : Find and fix a logical error in a Python loop that causes it to run endlessly. Correct the loop and briefly explain why the issue occurred and how it was resolved. Give both the original erroneous code and the corrected code.

Code :

```

1 #Find and fix a logical error in a Python loop that causes it to run endlessly.
2 #Correct the loop and briefly explain why the issue occurred and how it was resolved.
3 #Give both the original erroneous code and the corrected code.
4 # Original erroneous code
5 count = 0
6 while count < 5:
7     print("Count is:", count)
8     # Missing increment of count variable
9 # Corrected code
10 count = 0
11 while count < 5:
12     print("Count is:", count)
13     count += 1 # Incrementing count to avoid infinite loop
14 # Explanation:
15 # The issue occurred because the loop condition (count < 5) was never updated within the loop.
16 # As a result, the loop would run indefinitely since 'count' remained 0.

```

Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Count is: 0
Traceback (most recent call last):
  File "c:\Users\devis\OneDrive\Desktop\AI Assisted Coding\assignment7.3.py", line 7, in <module>
    print("Count is:", count)
    ~~~~~^~~~~~
KeyboardInterrupt
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:\Users\devis\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/devis/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
Count is: 0
Count is: 1
Count is: 2
Count is: 3
Count is: 4
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>

```

Explanation :

The loop ran endlessly because the count variable was never updated inside the loop. Since the condition `count < 5` always remained true, the loop never stopped. Adding an increment to count inside the loop updates the condition and allows the loop to terminate correctly.

TASK 3 :

Prompt : Identify the runtime error in a Python function that performs division. Fix the issue using proper error handling and explain how the solution prevents the program from crashing.

Code

```
assignment7.3.py > 📄 divide_numbers
1 #Identify the runtime error in a Python function that performs division.
2 #Fix the issue using proper error handling and explain how the solution prevents the program from
3 # Erroneous Code Snippet
4 def divide_numbers(num1, num2):
5     result = num1 / num2
6     return result
7 # Corrected Code Snippet
8 def divide_numbers(num1, num2):
9     try:
10         result = num1 / num2
11     except ZeroDivisionError:
12         return "Error: Division by zero is not allowed."
13     except TypeError:
14         return "Error: Invalid input type. Please provide numbers."
15     return result
16 # Example Usage
17 print(divide_numbers(10, 2)) # Output: 5.0
18 print(divide_numbers(10, 0)) # Output: Error: Division by zero is not allowed.
19 print(divide_numbers(10, 'a')) # Output: Error: Invalid input type. Please provide numbers.
20
```

Output :

```
PS C:\Users\devi\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/devi/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
5.0
Error: Division by zero is not allowed.
Error: Invalid input type. Please provide numbers.
PS C:\Users\devi\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

The erroneous code snippet does not handle the case where the second number (num2) is zero, which would raise a `ZeroDivisionError` and cause the program to crash. Additionally, if non-numeric types are passed as arguments, a `TypeError` would occur. The corrected code snippet includes a try-except block to catch these specific exceptions. This prevents the program from crashing by providing user-friendly error messages instead of terminating unexpectedly.

TASK – 04

Prompt : Find and correct the error in a Python class constructor where the required parameter is missing. Fix the class and briefly explain why this parameter is necessary. Give both the incorrect and corrected versions of the code.

Code :

```
1 #Find and correct the error in a Python class constructor where the required parameter is missing.
2 #Fix the class and briefly explain why this parameter is necessary.
3 #Give both the incorrect and corrected versions of the code.
4 # Incorrect version
5 class Car:
6     def __init__(self, make, model):
7         self.make = make
8         self.model = model
9 # Corrected version
10 class Car:
11     def __init__(self, make, model, year):
12         self.make = make
13         self.model = model
14         self.year = year
15 # Explanation:
16 # The 'year' parameter is necessary to fully define the Car object.
17 # Without it, we cannot capture all essential attributes of a car, which may be important for various operations or comparisons involving Car objects.
18 # Example usage:
19 my_car = Car("Toyota", "Corolla", 2020)
20 print(f"My car is a {my_car.year} {my_car.make} {my_car.model}.")
21
```

Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:\Users\devis\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/devis/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
My car is a 2020 Toyota Corolla.
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

The incorrect version of the class was missing the year parameter, so the Car object did not contain complete information. The corrected constructor includes the year parameter to properly initialize all essential attributes of a car. This allows each object to store full details and be used correctly in the program.

TASK – 05

Prompt : Detect and fix the error in a Python program caused by accessing an invalid list index. Implement a safe way to access list elements and explain the fix. Give both the erroneous code and the corrected code with an explanation.

Code :

```
#Detect and fix the error in a Python program caused by accessing an invalid list index.
#Implement a safe way to access list elements and explain the fix. Give both the erroneous code and the corrected code with an explanation.
# Erroneous Code
my_list = [10, 20, 30, 40, 50]
# Attempting to access an invalid index
print(my_list[10]) # This will raise an IndexError
# Corrected Code
my_list = [10, 20, 30, 40, 50]
# Safe way to access list elements
index = 10
if index < len(my_list):
    print(my_list[index])
else:
    print("Index out of range. Please provide a valid index.")
# Explanation of the Fix:
# The error in the original code occurs because we are trying to access an index (10) that does not exist in the list 'my_list', which only has 5 elements.
# To fix this, we added a check to see if the index is within the valid range of the list before attempting to access it.
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:\Users\devis\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/devis/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
Traceback (most recent call last):
  File "c:/Users/devis/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py", line 6, in <module>
    print(my_list[10]) # This will raise an IndexError
          ~~~~~~
IndexError: list index out of range
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:\Users\devis\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/devis/OneDrive/Desktop/AI Assisted Coding/assignment7.3.py"
Index out of range. Please provide a valid index.
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

The error occurred because the program tried to access an index that does not exist in the list. In the corrected code, a length check is added before accessing the list element. This ensures the index is within the valid range and prevents the program from crashing due to an IndexError.